

# NEXUS-COMPLY

## Complete API Functionality & Specification

SIH26155 — AI-Driven Multi-Vendor Network Security Compliance Auditor

**Scope:** Complete REST API layer for React frontend, Spring Boot backend, Cyber Engine, AI Training Service and MongoDB.

**Architecture principle:** Frontend never connects directly to MongoDB. Spring Boot exposes secured APIs, orchestrates the Cyber Engine, and communicates with the separate AI Training Service when unknown syntax requires AI assistance.

```
React / TypeScript
↓ HTTPS + JWT
Spring Boot REST API / Security Gateway
■■■ Identity & RBAC
■■■ Device / Configuration APIs
■■■ Cyber Engine APIs
■■■ Compliance / Finding / Risk APIs
■■■ Drift / What-If / Remediation APIs
■■■ Framework / Report / Dashboard APIs
■■■ AI Gateway APIs
↓
AI Training Service
↓
MongoDB
```

# 1. API Layer Scope

The API layer is the contract between the React frontend and the backend/security engine. It must expose all functionality required by the previously defined frontend, database and cyber-layer specifications.

| API domain              | Primary responsibility                 | Main consumers         |
|-------------------------|----------------------------------------|------------------------|
| Authentication & RBAC   | Identity, sessions, permissions        | Login, protected pages |
| Users & Settings        | Profiles, roles, settings              | Settings/admin         |
| Devices                 | Network inventory                      | Devices/dashboard/risk |
| Configurations          | Upload, versions, content, diff        | Configurations/audits  |
| Vendor Detection        | Identify vendor/platform               | Upload/audit workflow  |
| Parsing & Normalization | Convert raw config to canonical state  | Audit/configuration    |
| Audits                  | Start and track security audits        | Audit pages/dashboard  |
| Frameworks & Rules      | Controls, mappings, executable rules   | Compliance pages       |
| Findings & Evidence     | Issues and source proof                | Findings/evidence      |
| Risk                    | Contextual prioritization              | Risk center/dashboard  |
| Drift                   | Security-relevant configuration change | Drift page             |
| What-If                 | Temporary proposed-state analysis      | Simulation page        |
| Remediation             | Validated vendor-specific fixes        | Finding page           |
| AI Training             | Unknown syntax suggestion/approval     | AI Analyst             |
| Reports                 | PDF report lifecycle                   | Reports                |
| Dashboard               | Aggregated posture metrics             | Dashboard              |
| Audit Logs              | Traceable security/application actions | Admin/security         |

# 2. Technology and API Conventions

- REST/JSON APIs over HTTPS.
- Spring Boot owns authentication, authorization, validation and orchestration.
- MongoDB is accessed through the backend, never from React.
- JWT/session mechanism should be chosen and documented; access tokens must not contain secrets.
- Use consistent HTTP status codes.
- Use request IDs/correlation IDs for audit and troubleshooting.
- Paginate list endpoints.
- Use ISO-8601 timestamps.
- Use stable resource IDs rather than display names.
- Return deterministic status values and machine-readable error codes.
- Version the public API, e.g. /api/v1/..., if the team wants long-term compatibility.

Recommended base:  
/api/v1

Success:  
{ "data": ..., "requestId": "REQ-..." }

Error:  
{  
 "error": {  
 "code": "VALIDATION\_ERROR",  
 "message": "Human-readable message",  
 "details": [...]  
 },  
 "requestId": "REQ-..."  
}

# 3. Authentication & RBAC APIs

| Method | Endpoint           | Function          | Main data    |
|--------|--------------------|-------------------|--------------|
| POST   | /api/v1/auth/login | Authenticate user | users, roles |

| Method | Endpoint                  | Function               | Main data          |
|--------|---------------------------|------------------------|--------------------|
| POST   | /api/v1/auth/refresh      | Refresh access session | session/auth data  |
| POST   | /api/v1/auth/logout       | Invalidate session     | audit_logs/session |
| GET    | /api/v1/auth/me           | Current user           | users, roles       |
| GET    | /api/v1/users             | List users (admin)     | users              |
| POST   | /api/v1/users             | Create user            | users, roles       |
| GET    | /api/v1/users/{id}        | User details           | users              |
| PUT    | /api/v1/users/{id}        | Update user            | users              |
| PATCH  | /api/v1/users/{id}/status | Activate/deactivate    | users              |
| GET    | /api/v1/roles             | List roles             | roles              |
| POST   | /api/v1/roles             | Create/manage role     | roles              |
| GET    | /api/v1/permissions       | Permission catalog     | application config |

## Authentication functionality

- Credential validation
- Account status checks
- Session/JWT creation
- Refresh
- Logout/revocation strategy
- Current-user context
- Failed-login audit events

## RBAC functionality

- ADMIN
- SECURITY\_ANALYST
- AUDITOR
- OPERATOR/REVIEWER roles as needed
- Resource-level authorization
- AI mapping approval permission
- Remediation-management permission
- User-management permission

Authorization example:  
 AI\_MAPPING\_APPROVE → POST /api/v1/ai/mappings/{id}/approve  
 REMEDIATION\_MANAGE → remediation template administration  
 USER\_MANAGE → user/role administration

## 4. User Profile & Settings APIs

| Method | Endpoint                          | Function                  |
|--------|-----------------------------------|---------------------------|
| GET    | /api/v1/users/me                  | Profile                   |
| PUT    | /api/v1/users/me                  | Update profile            |
| PUT    | /api/v1/users/me/password         | Change password           |
| GET    | /api/v1/settings                  | Application/user settings |
| PUT    | /api/v1/settings                  | Update settings           |
| GET    | /api/v1/notifications/preferences | Notification preferences  |
| PUT    | /api/v1/notifications/preferences | Update preferences        |

Settings may include audit alerts, risk alerts, report preferences and other application options. Secrets such as database passwords and AI provider keys do not belong here.

## 5. Device APIs

| Method | Endpoint                            | Function                   |
|--------|-------------------------------------|----------------------------|
| GET    | /api/v1/devices                     | List/search/filter devices |
| POST   | /api/v1/devices                     | Register device            |
| GET    | /api/v1/devices/{id}                | Device details             |
| PUT    | /api/v1/devices/{id}                | Update device              |
| PATCH  | /api/v1/devices/{id}/status         | Activate/deactivate        |
| GET    | /api/v1/devices/{id}/posture        | Current security posture   |
| GET    | /api/v1/devices/{id}/risk           | Device risk                |
| GET    | /api/v1/devices/{id}/drift          | Device drift history       |
| GET    | /api/v1/devices/{id}/audits         | Device audits              |
| GET    | /api/v1/devices/{id}/configurations | Configuration history      |

```
POST /api/v1/devices
{
  "hostname": "RTR-01",
  "vendor": "Cisco",
  "platform": "IOS-XE",
  "model": "C9300",
  "osVersion": "17.x",
  "deviceType": "SWITCH",
  "criticality": "HIGH",
  "environment": "PRODUCTION"
}
```

- Validate vendor/platform values
- Assign stable device ID
- Store criticality/exposure context
- Record create/update actor
- Support filters: vendor, platform, status, criticality, environment

## 6. Configuration APIs

| Method | Endpoint                                               | Function                       |
|--------|--------------------------------------------------------|--------------------------------|
| POST   | /api/v1/configurations/upload                          | Secure upload + hash + version |
| GET    | /api/v1/configurations                                 | List configurations            |
| GET    | /api/v1/configurations/{id}                            | Configuration metadata         |
| GET    | /api/v1/configurations/{id}/content                    | Controlled raw content         |
| GET    | /api/v1/configurations/{id}/versions                   | Version history                |
| GET    | /api/v1/configurations/versions/{versionId}            | Version details                |
| GET    | /api/v1/configurations/versions/{versionId}/normalized | Canonical state                |
| GET    | /api/v1/configurations/versions/{versionId}/diff       | Compare versions               |
| POST   | /api/v1/configurations/versions/{versionId}/audit      | Start audit                    |

### Upload workflow

Upload → authentication → size/type validation → content validation → SHA-256 → device association → version creation → vendor detection → parser → normalized state

- Reject unsupported/oversized files
- Detect duplicate hash
- Never execute uploaded commands
- Keep raw configuration protected
- Return asynchronous job/audit reference when processing is long

## 7. Vendor Detection APIs

| Method | Endpoint                      | Function                            |
|--------|-------------------------------|-------------------------------------|
| POST   | /api/v1/cyber/detect-vendor   | Detect vendor/platform from content |
| GET    | /api/v1/cyber/detections/{id} | Detection result                    |

| Method | Endpoint                              | Function                      |
|--------|---------------------------------------|-------------------------------|
| POST   | /api/v1/cyber/detections/{id}/confirm | Human correction/confirmation |

Response:

```
{
  "vendor": "Cisco",
  "platform": "IOS-XE",
  "confidence": 0.97,
  "status": "DETECTED",
  "method": "SYNTAX_FINGERPRINT"
}
```

A detection must support DETECTED, UNCERTAIN and UNSUPPORTED/UNKNOWN outcomes.

## 8. Parser APIs

| Method | Endpoint                                  | Function                  |
|--------|-------------------------------------------|---------------------------|
| POST   | /api/v1/cyber/parse                       | Parse configuration       |
| GET    | /api/v1/cyber/parse-jobs/{jobId}          | Parser status             |
| GET    | /api/v1/cyber/parse-jobs/{jobId}/errors   | Safe parser diagnostics   |
| GET    | /api/v1/cyber/parse-jobs/{jobId}/unknowns | Unknown syntax constructs |

The parser API should return structured facts and source-location metadata. It must not return a framework PASS/FAIL; that belongs to the compliance engine.

## 9. Normalization APIs

| Method | Endpoint                                          | Function                                 |
|--------|---------------------------------------------------|------------------------------------------|
| POST   | /api/v1/cyber/normalize                           | Create canonical security state          |
| GET    | /api/v1/normalized-configurations/{id}            | Retrieve canonical state                 |
| GET    | /api/v1/configurations/versions/{id}/normalized   | Canonical state for version              |
| GET    | /api/v1/normalized-configurations/{id}/source-map | Canonical fact → source evidence mapping |

Canonical example:

```
{
  "security": {
    "ssh": {"enabled": true, "version": 2},
    "telnet": {"enabled": false}
  },
  "authentication": {"aaa": true},
  "logging": {"syslog": true}
}
```

## 10. Framework & Control APIs

| Method | Endpoint                         | Function                  |
|--------|----------------------------------|---------------------------|
| GET    | /api/v1/frameworks               | List supported frameworks |
| GET    | /api/v1/frameworks/{id}          | Framework details         |
| GET    | /api/v1/frameworks/{id}/controls | Controls                  |
| GET    | /api/v1/controls/{id}            | Control details           |
| GET    | /api/v1/controls/{id}/rules      | Executable rules          |
| GET    | /api/v1/rules/{id}               | Rule details              |
| GET    | /api/v1/framework-mappings       | Cross-framework mappings  |
| GET    | /api/v1/framework-mappings/{id}  | Mapping details           |

Framework records cover CIS Benchmarks, NIST SP 800-53, DISA STIG and ISO/IEC 27001 technical-evidence mappings. Version and applicability metadata are essential.

## 11. Compliance Audit APIs

| Method | Endpoint       | Function           |
|--------|----------------|--------------------|
| POST   | /api/v1/audits | Create/start audit |
| GET    | /api/v1/audits | List/filter audits |

| Method | Endpoint                              | Function                |
|--------|---------------------------------------|-------------------------|
| GET    | /api/v1/audits/{id}                   | Audit details           |
| GET    | /api/v1/audits/{id}/status            | Processing stage/status |
| POST   | /api/v1/audits/{id}/cancel            | Cancel if supported     |
| POST   | /api/v1/audits/{id}/rerun             | Re-run audit            |
| GET    | /api/v1/audits/{id}/summary           | Audit summary           |
| GET    | /api/v1/audits/{id}/framework-results | Framework results       |
| GET    | /api/v1/audits/{id}/findings          | Audit findings          |
| GET    | /api/v1/audits/{id}/risk              | Audit risk              |
| POST   | /api/v1/audits/{id}/report            | Generate report         |

Audit state:

QUEUED → DETECTING → PARSING → NORMALIZING → UNKNOWN\_REVIEW → CHECKING → RISK\_CALCULATION → COMPLETED / FAILED

The frontend uses status polling or an event mechanism to display audit progress.

## 12. Compliance Rule Execution APIs

| Method | Endpoint                                      | Function                                |
|--------|-----------------------------------------------|-----------------------------------------|
| POST   | /api/v1/compliance/evaluate                   | Evaluate selected version against rules |
| POST   | /api/v1/compliance/evaluate/control/{id}      | Evaluate one control                    |
| POST   | /api/v1/compliance/evaluate/framework/{id}    | Evaluate framework                      |
| GET    | /api/v1/compliance/results/{auditId}          | Results                                 |
| GET    | /api/v1/compliance/results/{auditId}/summary  | Counts/scores                           |
| GET    | /api/v1/compliance/results/{auditId}/unknowns | UNKNOWN results                         |

Compliance results must distinguish PASS, FAIL, UNKNOWN, NOT\_APPLICABLE and ERROR. Rule errors must never be converted to PASS.

## 13. Findings APIs

| Method | Endpoint                          | Function                                |
|--------|-----------------------------------|-----------------------------------------|
| GET    | /api/v1/findings                  | List/filter findings                    |
| GET    | /api/v1/findings/{id}             | Finding details                         |
| PATCH  | /api/v1/findings/{id}/status      | Change finding status                   |
| PATCH  | /api/v1/findings/{id}/severity    | Controlled severity update if permitted |
| GET    | /api/v1/findings/{id}/history     | Finding lifecycle history               |
| GET    | /api/v1/findings/{id}/related     | Related findings/control results        |
| POST   | /api/v1/findings/{id}/acknowledge | Acknowledge                             |
| POST   | /api/v1/findings/{id}/resolve     | Resolve                                 |

```
Finding:
{
  "status": "FAIL",
  "severity": "HIGH",
  "framework": "CIS",
  "controlId": "...",
  "ruleId": "...",
  "expected": "SSH version 2",
  "actual": "SSH version 1"
}
```

## 14. Evidence APIs

| Method | Endpoint                            | Function                         |
|--------|-------------------------------------|----------------------------------|
| GET    | /api/v1/findings/{id}/evidence      | All evidence                     |
| GET    | /api/v1/evidence/{id}               | Evidence details                 |
| GET    | /api/v1/evidence/{id}/source        | Source context                   |
| GET    | /api/v1/evidence/{id}/configuration | Associated configuration/version |

- Line number and source text
- Expected vs actual
- Canonical fact
- Rule/control reference
- Configuration version
- Explanation
- Evidence quality/status

## 15. Risk APIs

| Method | Endpoint                           | Function                 |
|--------|------------------------------------|--------------------------|
| GET    | /api/v1/risk                       | Global risk summary      |
| GET    | /api/v1/risk/findings              | Finding-level risk       |
| GET    | /api/v1/risk/devices               | Device risk              |
| GET    | /api/v1/risk/devices/{id}          | Device risk details      |
| GET    | /api/v1/risk/trend                 | Historical risk          |
| POST   | /api/v1/risk/recalculate/{auditId} | Recalculate risk         |
| GET    | /api/v1/risk/{findingId}           | Risk factors for finding |

Risk must preserve calculation factors such as finding severity, asset criticality, exposure and any other documented factors. Severity and contextual risk remain distinct.

## 16. Drift APIs

| Method | Endpoint                             | Function                           |
|--------|--------------------------------------|------------------------------------|
| GET    | /api/v1/drift                        | Global drift events                |
| GET    | /api/v1/drift/{id}                   | Drift details                      |
| GET    | /api/v1/devices/{id}/drift           | Device drift timeline              |
| POST   | /api/v1/drift/compare                | Compare two configuration versions |
| GET    | /api/v1/drift/{id}/affected-controls | Affected controls                  |
| GET    | /api/v1/drift/{id}/risk-impact       | Risk before/after                  |

```
Version N + Version N+1
↓
Canonical state diff
↓
Security-relevant changes
↓
Affected controls + risk delta
```

## 17. What-If Simulation APIs

| Method | Endpoint                       | Function                                     |
|--------|--------------------------------|----------------------------------------------|
| POST   | /api/v1/simulations            | Create simulation                            |
| GET    | /api/v1/simulations            | List simulations                             |
| GET    | /api/v1/simulations/{id}       | Simulation result                            |
| POST   | /api/v1/simulations/{id}/rerun | Recalculate                                  |
| DELETE | /api/v1/simulations/{id}       | Delete temporary simulation if policy allows |

```
POST /api/v1/simulations
{
  "deviceId": "DEV-001",
  "baseVersionId": "VER-004",
  "changes": [
    { "field": "security.telnet.enabled", "newValue": true }
  ]
}
```

The simulation operates on a temporary state and must never mutate the real configuration/version history.

## 18. Remediation APIs

| Method | Endpoint                                | Function                 |
|--------|-----------------------------------------|--------------------------|
| GET    | /api/v1/remediation/templates           | List validated templates |
| GET    | /api/v1/remediation/templates/{id}      | Template details         |
| GET    | /api/v1/findings/{id}/remediation       | Recommended remediation  |
| POST   | /api/v1/findings/{id}/remediation/plan  | Create remediation plan  |
| POST   | /api/v1/remediation/plans/{id}/validate | Validate plan            |
| GET    | /api/v1/remediation/plans/{id}          | Plan details             |
| POST   | /api/v1/remediation/plans/{id}/verify   | Verify after a new audit |

- Select by vendor/platform/finding type
- Ordered steps
- Validation condition
- Approval requirement
- Human review
- Verification audit

For the prototype, remediation should be guidance/templates, not autonomous production command execution.

## 19. AI Training Service APIs

The AI Training Service is separate from the deterministic Cyber Engine. Spring Boot acts as the controlled gateway/orchestrator.

| Method | Endpoint                         | Function                    |
|--------|----------------------------------|-----------------------------|
| POST   | /api/v1/ai/analyze               | Analyze unknown syntax      |
| GET    | /api/v1/ai/jobs/{id}             | AI analysis status          |
| GET    | /api/v1/ai/jobs/{id}/suggestions | Suggestions                 |
| GET    | /api/v1/ai/mappings              | Mapping library             |
| GET    | /api/v1/ai/mappings/{id}         | Mapping details             |
| POST   | /api/v1/ai/mappings/{id}/approve | Human approval              |
| POST   | /api/v1/ai/mappings/{id}/reject  | Reject suggestion           |
| POST   | /api/v1/ai/mappings/{id}/test    | Test mapping against sample |
| GET    | /api/v1/ai/mappings/{id}/usage   | Reuse/history               |

```
AI request:
{
  "vendor": "UnknownVendor",
  "platform": "X-OS",
  "rawSyntax": "set secure-admin ssh protocol v2",
  "context": {"sourceLine": 42}
}
```

```
AI suggestion:
{
  "canonicalField": "security.ssh.version",
  "value": 2,
  "confidence": 0.91,
  "status": "REVIEW_REQUIRED"
}
```

- AI suggests semantic mappings
- Human approves/rejects
- Approved mappings are persisted
- Cyber Engine re-runs deterministic rules
- AI failure never prevents deterministic auditing
- Prompt injection and untrusted configuration content must be handled safely

## 20. Dashboard APIs

| Method | Endpoint                     | Function                      |
|--------|------------------------------|-------------------------------|
| GET    | /api/v1/dashboard/summary    | Overall posture               |
| GET    | /api/v1/dashboard/compliance | Compliance metrics            |
| GET    | /api/v1/dashboard/findings   | Finding metrics               |
| GET    | /api/v1/dashboard/risk       | Risk metrics                  |
| GET    | /api/v1/dashboard/drift      | Drift metrics                 |
| GET    | /api/v1/dashboard/activity   | Recent activity               |
| GET    | /api/v1/dashboard/frameworks | Framework comparison/coverage |

Dashboard APIs should aggregate from devices, audits, findings, risk, drift and audit logs rather than duplicating authoritative data.

## 21. Reports APIs

| Method | Endpoint                        | Function               |
|--------|---------------------------------|------------------------|
| POST   | /api/v1/reports                 | Create report          |
| GET    | /api/v1/reports                 | List reports           |
| GET    | /api/v1/reports/{id}            | Report metadata/status |
| GET    | /api/v1/reports/{id}/preview    | Preview data           |
| GET    | /api/v1/reports/{id}/download   | Download PDF           |
| POST   | /api/v1/reports/{id}/regenerate | Regenerate             |

- Executive summary
- Compliance results
- Findings/evidence
- Framework mappings
- Risk assessment
- Drift where relevant
- Remediation
- Audit/configuration version references

## 22. Audit Log APIs

| Method | Endpoint                                | Function          |
|--------|-----------------------------------------|-------------------|
| GET    | /api/v1/audit-logs                      | Search audit logs |
| GET    | /api/v1/audit-logs/{id}                 | Log detail        |
| GET    | /api/v1/audit-logs/resource/{type}/{id} | Resource history  |

- Login/logout
- Configuration upload
- Device changes
- Audit start/completion
- Finding status changes
- AI mapping approval/rejection
- Remediation actions
- Report generation
- User/role changes

## 23. API Search, Filtering and Pagination

```
GET /api/v1/findings?page=0&size=20&severity;=HIGH&status;=OPEN&framework;=CIS
GET /api/v1/devices?page=0&size=20&vendor;=Cisco&criticality;=HIGH
GET /api/v1/audits?deviceId=DEV-001&status;=COMPLETED
```

- All large lists should paginate
- Use server-side filtering for findings/devices/audits
- Allow sorting by supported fields
- Return total/hasNext metadata
- Do not load entire configuration histories into dashboard requests

## 24. Standard HTTP Status Strategy

| Status  | Use                                                  |
|---------|------------------------------------------------------|
| 200     | Successful GET/update/result                         |
| 201     | Resource created                                     |
| 202     | Accepted asynchronous processing                     |
| 204     | Successful action with no response body              |
| 400     | Malformed request                                    |
| 401     | Missing/invalid authentication                       |
| 403     | Authenticated but not authorized                     |
| 404     | Resource not found                                   |
| 409     | Conflict, duplicate version, invalid lifecycle state |
| 413     | Upload too large                                     |
| 422     | Semantically invalid configuration/request           |
| 429     | Rate limit                                           |
| 500     | Unexpected server error                              |
| 502/503 | Dependent service unavailable                        |

## 25. Common Error Contract

```
{
  "error": {
    "code": "CONFIG_PARSE_FAILED",
    "message": "Configuration could not be parsed.",
    "details": [
      { "line": 184, "reason": "Unknown syntax" }
    ]
  },
  "requestId": "REQ-123"
}
```

- Never return stack traces
- Do not expose database paths
- Do not expose secrets
- Use stable error codes for frontend handling
- Log detailed diagnostics server-side with controlled access

## 26. Asynchronous Job Model

Parsing, AI analysis, audits and PDF generation may be asynchronous. A common job pattern prevents long HTTP requests.

```
POST → 202 Accepted → jobId
GET /jobs/{jobId}
↓
QUEUED → RUNNING → COMPLETED / FAILED / CANCELLED
```

| Job type    | Reason                                |
|-------------|---------------------------------------|
| AUDIT       | Multiple processing stages            |
| PARSER      | Large configuration                   |
| AI_ANALYSIS | External/model latency                |
| REPORT      | PDF generation                        |
| SIMULATION  | Potentially expensive rule evaluation |

## 27. Complete API-to-Cyber Workflow

1. POST /configurations/upload
2. POST /cyber/detect-vendor
3. POST /cyber/parse
4. POST /cyber/normalize
5. POST /audits
6. compliance engine evaluates rules
7. GET /audits/{id}/findings
8. GET /findings/{id}/evidence
9. GET /risk/{findingId}
10. GET /drift or compare versions
11. POST /simulations
12. GET /findings/{id}/remediation
13. POST /ai/analyze when syntax is unknown
14. approve mapping if validated
15. re-run audit
16. POST /reports
17. dashboard reflects final state

## 28. API-to-MongoDB Mapping

| API domain     | Primary collections                    |
|----------------|----------------------------------------|
| Auth/RBAC      | users, roles, audit_logs               |
| Devices        | devices, audit_logs                    |
| Configurations | configurations, configuration_versions |
| Normalization  | normalized_configurations              |
| Frameworks     | frameworks, controls, compliance_rules |
| Audits         | audits                                 |
| Findings       | findings                               |
| Evidence       | evidence                               |
| Risk           | risk_assessments                       |
| Drift          | drift_events                           |
| AI             | ai_mappings, audit_logs                |
| Remediation    | remediation_templates                  |
| Simulation     | what_if_simulations                    |
| Reports        | reports                                |

## 29. API Security Requirements

- Every protected endpoint requires authentication.
- Every sensitive endpoint requires permission checks.
- Never rely on React route protection as authorization.
- Validate every request body and path/query parameter.
- Use allowlists for vendor/platform/framework/status enums.
- Apply upload size and parsing limits.
- Rate-limit authentication, upload, AI and expensive operations.
- Use request IDs and security audit logs.
- Never expose MongoDB directly to clients.
- Protect raw configuration and evidence endpoints.
- Do not execute configuration content.
- Do not execute AI-generated commands.
- Sanitize data passed to AI and defend against prompt injection.
- Apply timeouts when calling the AI service.
- Fail safely when AI service is unavailable.
- Use least-privilege service credentials.

- Keep secrets in environment/secret management.

## 30. API Validation Rules

| Area        | Important validation                                      |
|-------------|-----------------------------------------------------------|
| Devices     | Hostname, vendor/platform, enum values, criticality       |
| Upload      | File size, type, name, content, device association        |
| Audit       | Existing version, framework availability, valid lifecycle |
| Findings    | Allowed statuses and transitions                          |
| Risk        | Valid source finding/device, documented factors           |
| Drift       | Two valid versions of same compatible device              |
| Simulation  | Valid base version and canonical field                    |
| AI          | Input length, source context, approval permissions        |
| Remediation | Template applicability, approval state                    |
| Reports     | Existing audit and accessible data                        |

## 31. Frontend Page → API Mapping

| Frontend page            | Primary APIs                                 |
|--------------------------|----------------------------------------------|
| Login                    | auth/*                                       |
| Dashboard                | dashboard/*, devices, audits, findings, risk |
| Devices                  | devices/*                                    |
| Configurations           | configurations/*, cyber/detect-vendor        |
| Normalized Configuration | normalized-configurations/*                  |
| Audits                   | audits/*                                     |
| Findings                 | findings/*, evidence/*                       |
| Risk Center              | risk/*                                       |
| Drift Intelligence       | drift/*                                      |
| What-If                  | simulations/*                                |
| AI Analyst               | ai/*                                         |
| Reports                  | reports/*                                    |
| Frameworks & Controls    | frameworks/*, controls/*, rules/*            |
| Settings/Profile         | users/me, settings, notifications            |

## 32. API Implementation Phases

| Phase             | Build                                                       |
|-------------------|-------------------------------------------------------------|
| 1 — Foundation    | Auth, RBAC, common errors, request IDs, devices             |
| 2 — Configuration | Upload, hash/version, content, history, diff                |
| 3 — Cyber core    | Detection, parser, normalization, audit orchestration       |
| 4 — Compliance    | Frameworks, controls, rules, compliance results             |
| 5 — Findings      | Findings, evidence, lifecycle                               |
| 6 — Risk/action   | Risk, remediation, audit logs                               |
| 7 — Intelligence  | Drift, what-if                                              |
| 8 — AI            | AI gateway, analysis, mapping approval/reuse                |
| 9 — Output        | Reports + dashboard aggregations                            |
| 10 — Hardening    | Security tests, rate limits, performance, integration tests |

## 33. API Testing Strategy

- Unit-test controllers, validators and service methods.
- Integration-test Spring Boot + MongoDB.

- Test every authorization boundary.
- Test upload edge cases and malicious filenames.
- Test parser and normalization API contracts with known/unknown configs.
- Test deterministic compliance results.
- Test PASS/FAIL/UNKNOWN/N/A/ERROR handling.
- Test finding status transitions.
- Test risk reproducibility.
- Test drift version compatibility.
- Test what-if isolation from real versions.
- Test AI approval/rejection and AI outage fallback.
- Test report generation against audit data.
- Test pagination/filtering.
- Test rate limits and timeout behavior.
- Run an end-to-end four-vendor scenario.

## 34. Complete API Inventory

- Authentication: 4 core endpoints
- Users/RBAC: user, role and permission management
- Settings/profile: profile, password and notification settings
- Devices: CRUD + posture/risk/drift/audits/configurations
- Configurations: upload + metadata + versions + normalized state + diff + audit
- Vendor detection: detect + retrieve + confirm
- Parser: parse + job status + errors + unknowns
- Normalization: create + retrieve + source map
- Frameworks/controls/rules: catalog and mapping retrieval
- Audits: create/list/detail/status/cancel/rerun/summary/framework/findings/risk/report
- Compliance: evaluate + framework/control + results/summary/unknowns
- Findings: list/detail/status/history/related/acknowledge/resolve
- Evidence: list/detail/source/configuration
- Risk: global/finding/device/trend/recalculate
- Drift: list/detail/compare/affected-controls/risk-impact
- What-if: create/list/detail/rerun/delete
- Remediation: templates/recommendation/plans/validate/verify
- AI Training: analyze/jobs/suggestions/mappings/approve/reject/test/usage
- Dashboard: summary/compliance/findings/risk/drift/activity/frameworks
- Reports: create/list/detail/preview/download/regenerate
- Audit logs: search/detail/resource history

## 35. Definition of Done

- Every frontend page has a documented API contract.
- Every protected endpoint has authentication and authorization rules.
- Every endpoint has validation and error behavior.
- Long-running work has a job/status mechanism.
- API resources map to MongoDB collections without duplicating authoritative state unnecessarily.

- Cyber decisions remain deterministic.
- AI remains an assisting service with human approval for reusable mappings.
- Raw configurations and evidence are protected.
- Dashboard data is derived from authoritative security data.
- All critical APIs have unit + integration + authorization tests.
- At least one complete four-vendor workflow works end-to-end.
- OpenAPI/Swagger documentation is generated from the Spring Boot implementation.

## 36. Recommended Spring Boot Package Structure

```
com.nexuscomply
  ■■■ auth
  ■■■ user
  ■■■ device
  ■■■ configuration
  ■■■ detection
  ■■■ parser
  ■■■ normalization
  ■■■ audit
  ■■■ framework
  ■■■ compliance
  ■■■ finding
  ■■■ evidence
  ■■■ risk
  ■■■ drift
  ■■■ simulation
  ■■■ remediation
  ■■■ ai
  ■■■ report
  ■■■ dashboard
  ■■■ auditlog
  ■■■ common
  ■■■ exception
  ■■■ validation
  ■■■ security
  ■■■ response
```

## 37. Final API Architecture

```
React
↓
/api/v1/*
↓
Spring Security
↓
Controllers
↓
Application Services
  ■■■ Cyber Engine
  ■ ■■■ Detection
  ■ ■■■ Parser
  ■ ■■■ Normalization
  ■ ■■■ Compliance
  ■ ■■■ Evidence
  ■ ■■■ Risk
  ■ ■■■ Drift
  ■ ■■■ What-If
  ■ ■■■ Remediation
  ■
  ■■■ AI Gateway
↓
AI Training Service
↓
MongoDB / AI mappings

Spring Data MongoDB
↓
MongoDB
```

## 38. Final Recommendation

Build the API layer as the stable contract that connects the frontend, Cyber Engine, AI Training Service and MongoDB. The most important boundary is that the **Cyber Engine owns deterministic security/compliance decisions**, while the **AI Training Service owns semantic suggestions and reusable mappings**. The API layer orchestrates both but should never allow AI output to bypass validation, authorization, evidence creation or deterministic compliance rules.

For the first working vertical slice, implement: authentication → device → configuration upload/version → vendor detection → one parser → normalization → audit → a small validated compliance rule set → finding/evidence → dashboard. Then expand the same API contract to risk, drift, what-if, remediation, AI and reports.